오픈소스 반도체 설계 및 제작 종합설계 #4

우주 침략자 게임의 FPGA 에뮬레이션 환경 구축과 면적 검토

2021-17385
산업공학과 김예원

[1] 개발 도구 설치

이번 주 과제는 다음 주에 배포되는 FPGA 검증 보드를 바로 사용할 수 있도록 개발 도구를 설치하고, Q.13과 Q.14의 Talks, simulation, emulation, ETRI050 디렉토리를 차례로 실행하며 각 단계에서 Makefile이 어떤 도구를 부르는지 확인하는 것이다. 실습 저장소를 10월 14일 버전으로 갱신한 뒤, WSL(Ubuntu)에 Arduino 개발 도구(arduino-cli, Raspberry Pi Pico 보드 패키지, U8g2 라이브러리)와 Quartus Prime Standard 25.1(Cyclone IV 장치)을 설치하였다. Quartus는 30일 평가판으로 사용한다.

[2] Talks 디렉토리 검토와 면적 평가 검증

Q.14의 Talks 디렉토리에는 AI(Gemini)와 나눈 여섯 개의 질의응답이 있다. Q.13 폴더에는 Talks 디렉토리가 없어 Q.14의 대화만 검토하였다. 점수 기능을 뺀 RTL(No_Score)을 ETRI 0.5μm 공정의 내 칩 MPW로 제작할 때 면적이 충분한지 묻는 것으로 시작한다. AI는 1.5mm×1.5mm 칩에서 패드를 제외한 코어 면적을 약 1.0mm²로 보고, 이 RTL이 12,000~15,000 게이트로 허용치(약 10,000 게이트)를 넘을 위험이 있다고 답하였다. 그 원인으로는 5×5 인베이더를 검사하는 for 루프가 하드웨어로 25번 복제되고, 루프 안의 곱셈(열 번호×12, 행 번호×8)이 곱셈기로 합성된다는 점을 들었다. 이어서 AI는 충돌 검사를 1클럭에 인베이더 한 마리씩 하도록 바꾸고(chk_idx 카운터 추가) 곱셈을 시프트와 덧셈으로 바꾼 코드(v_1)를 제시하며, 면적이 약 80% 줄어 2,000 게이트 수준이 된다고 하였다.

AI가 제시한 코드는 그대로 사용할 수 없었다. v_1에서 변수 c, t_invader_x, t_invader_y가 선언되지 않았다는 것을 지적하였고, AI는 선언을 추가한 v_2를 내놓았다. 다시 loop_r의 비트 범위 문제를 지적하였다. loop_r는 3비트 부호 없는 변수인데, for 문에서 0에서 1을 빼면 7이 되므로 'loop_r가 0 이상'이라는 조건이 항상 참이 되어 루프가 끝나지 않는다. 이에 AI는 loop_r를 정수(integer)형으로 바꾼 v_3를 제시하고, 이 코드가 1,800~2,200 게이트, 0.08~0.1mm²로 코어의 약 10%만 차지한다고 평가하였다. 대화에서 코드의 오류를 찾아낸 것은 사람이었고, AI는 지적받은 부분만 수정하였다.

AI가 제시한 면적 수치를 확인하기 위해 각 버전을 ETRI050 디렉토리와 같은 방법(qflow synthesize -T etri050, 내부에서 yosys 사용)으로 합성하였다. 처음에는 qflow가 합성 도구가 정의되지 않았다는 오류로 멈추었는데, 설치된 qflow의 기본 합성 도구 설정이 비어 있었기 때문이다. 프로젝트 설정 파일(project_vars.sh)에 set synthesis_tool = yosys 한 줄을 추가하여 해결하였다. 합성된 셀 면적의 합을 ETRI 표준 셀 라이브러리의 2입력 NAND 게이트 면적(216μm²)으로 나누어

게이트 수(GE)로 환산하였고, 결과는 표 1과 같다.

표 1. 버전별 합성 결과 (ETRI 0.5μm 표준 셀, 배치 전 셀 면적 합)

버전	셀 수	플립플롭	셀 면적 (μm²)	게이트 수 (GE)	No_Score 대비
0 Score	2,107	118	639,828	2,962	+22.7%
1 No_Score (원본)	1,688	103	521,271	2,413	기준
2 v_1	–	–	–	–	합성 미완료
3 v_2	–	–	–	–	합성 미완료
4 v_3 (AI 재설계)	1,760	108	551,754	2,554	+5.8%

원본(No_Score)은 약 2,410 게이트로, AI가 추정한 12,000~15,000 게이트의 1/5~1/6이었다. AI가 재설계한 v_3는 약 2,550 게이트로 원본보다 오히려 5.8% 컸다. 원본의 합성 기록을 확인한 결과, for 루프 안의 곱셈은 루프가 펼쳐지면서 행과 열 번호가 상수가 되어 곱셈기로 합성되지 않았다. 남은 곱셈 셀은 변수와 상수를 곱하는 한 곳(shoot_col×12)뿐이었다. 또한 AI는 충돌 검사 루프만 순차 방식으로 바꾸었고, 매 화소마다 25마리를 동시에 비교해야 하는 화면 그리기 루프는 그대로 두었다. 여기에 chk_idx 카운터와 이를 행과 열로 나누는 비교·뺄셈 회로가 더해져 면적이 늘어난 것으로 판단하였다.

AI가 v_3에 대해 제시한 게이트 수는 실측과 비슷하였으나, 게이트 하나의 면적을 40~50μm²로 실제(216μm²)의 1/4~1/5로 잡아 개선판의면적(0.08~0.1mm²)을 크게 낮게 평가하였다. 실제 셀 면적은 0.55mm²로, AI가 말한 코어 면적의 절반을 넘는다. 반대로 원본도 면적 초과 위험은 없었으며, 면적을 실제로 줄인 변경은 점수 기능 삭제(Score에서 No_Score로, 18.5% 감소)였다.

v_1과 v_2는 Verilator의 lint 검사에서 각각 선언 누락과 무한 루프(INFINITELOOP) 오류가 발생하였고, yosys 합성도 끝나지 않았다. v_2를 합성할 때는 yosys가 메모리를 모두 사용하여 WSL이 재시작되었고, 이후에는 프로세스의 메모리 사용량을 2.5GB로 제한(ulimit)하고 합성하였다. 대화에서 지적한 두 오류가 실제로 합성을 막는 오류였음을 도구로 확인하였다.

[3] simulation: RTL 버전에 따른 차이

Q.14의 simulation 디렉토리에서 make를 실행하면 다섯 개의 RTL(0 Score, 1 No_Score, 2 v_1, 3 v_2, 4 v_3) 중 하나를 골라 링크한다. 2~4번은 [2]의 대화에서 나온 코드이다. 0번을 Q.13의 최종본(4_Ok)과 비교하면 출력 핀이 바뀌었다.

Q.13은 화소마다 좌표(lcd_x 7비트, lcd_y 6비트), 쓰기 신호(lcd_write_en), 화소 값(lcd_data)을 내보내 출력이 15비트였다. Q.14는 좌표를 내보내지 않고, 한 화면이 시작될 때 v_sync를 1로 만든 뒤 화소 값을 정해진 순서대로 내보낸다. 출력은 clk_o, v_sync, lcd_data의 3비트로 줄었다. 내부에서도 별도의 화면 스캔 카운터를 없애고, 15비트 game_tick_counter의 하위 7비트와 그다음 6비트를 각각 화면의 x, y 좌표로 사용한다. 칩의 핀 수가 정해져 있으므로(ETRI050 디렉토리는 28핀

패드 프레임을 사용한다) 출력 핀을 줄이기 위한 변경으로 판단하였다. 테스트벤치의 화면 모델도 좌표를 입력받지 않고 내부 카운터로 화소 위치를 세다가 v_sync에서 0으로 되돌리도록 바뀌었다.

1번은 0번에서 점수 기능을 삭제한 버전이다. 숫자 열 개의 글꼴을 담은 font_rom(80바이트), 두 자리 점수 레지스터(score_bcd), 점수를 그리는 회로가 빠졌고, 난수용 카운터(rand_counter)는 game_tick_counter의 하위 8비트를 대신 쓰도록 바뀌었다.

0, 1, 4번을 차례로 링크하여 VCD_TRACE=NO make build로 빌드하고 실행하였다. 2번과 3번은 lint 오류로 빌드되지 않는다. 화면 모델의 창이 128×64 화소로 작아, 지난주 Q.13과 같이 512×256 창에 4배로 확대하여 그리도록 sc_glcd128x64_TLM.h의 두 줄을 수정하였다. 0번에서는 왼쪽 위에 점수가 표시되었고 외계인을 맞힐 때마다 값이 올라갔다(그림 1). 1번에서는 점수 표시 자리가 비어 있었고, 목숨 표시와 나머지 동작은 0번과 같았다(그림 2). 4번은 1번과 화면이 같았으며, 외계인을 맞혔을 때 외계인과 총알이 1번과 마찬가지로 즉시 사라졌다(그림 3).

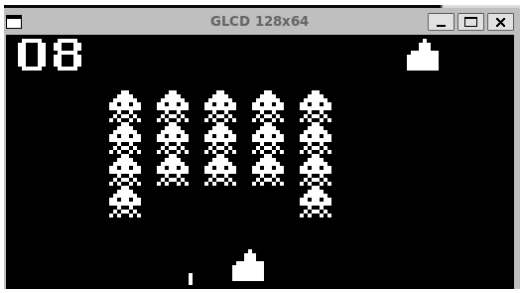


그림 1. 0번 Score 실행 화면 (왼쪽 위 점수 08, 오른쪽 위 남은 목숨 1개)

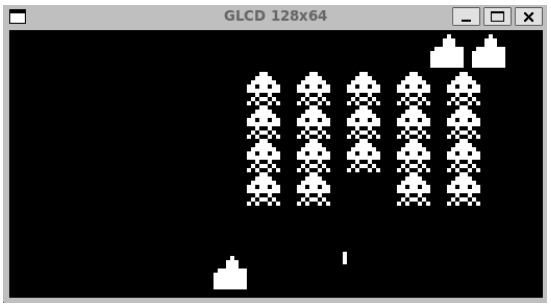


그림 2. 1번 No_Score 실행 화면 (점수 표시 없음)

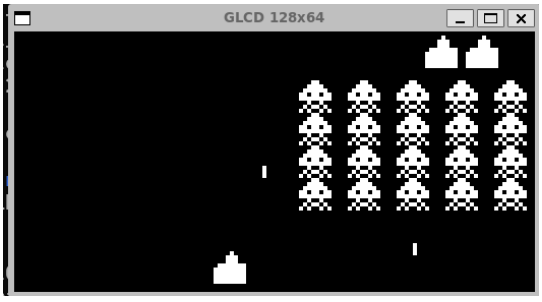


그림 3. 4번 v_3 실행 화면 (1번과 같은 화면 구성)

4번을 실행할 때 반응이 조금 빠르다고 느꼈으나, 측정한 결과 차이는 없었다. 저장소 밖의 복사본에서 화면 없이 30초씩 실행하며 1초에 그린 프레임 수를 세었다. 테스트벤치에는 50만 클럭

마다 발사 버튼을 누르는 코드를 넣어, 총알이 날아다니는 상태에서 측정하였다. 두 번 측정한 결과 0번은 51.6, 42.6 프레임/초, 1번은 53.0, 51.6 프레임/초, 4번은 49.8, 44.0 프레임/초였다. 같은 버전에서도 회차마다 최대 약 20%의 차이가 났고, 버전 사이의 차이는 그 범위 안에 있었다. 1번과 4번은 게임 속도를 정하는 회로가 같다. 인베이더와 총알은 32,768클럭마다 한 칸 움직이고, 대포는 한 화면이 끝날 때 한 번 버튼을 확인한다. 4번의 충돌 검사는 25클럭에 인베이더 25마리를 한 바퀴 돌므로, 총알이 한 칸에 머무는 동안 같은 인베이더를 약 1,300번 검사한다. 따라서 순차 검사로 바꾸어도 명중 판정의 지연은 화면에 나타나지 않으며, 체감한 차이는 실행 당시 PC 부하에 따른 시뮬레이션 속도 변동으로 판단하였다.

[4] emulation: FPGA 검증 환경 빌드

emulation은 PC, Pi Pico, FPGA의 세 부분으로 구성된다. **PC**에서는 simulation과 같은 SystemC 테스트벤치와 화면 모델이 실행되는데, EMULATED_CO_SIM을 정의하여 빌드하면 Verilator로 변환한 RTL 대신 USB 시리얼로 Pico와 통신한다. **Pico의 펌웨어(PSCE-MI)**는 PC에서 받은 버튼 입력을 FPGA로 전달하고, FPGA의 출력을 PC로 돌려준다. **FPGA**에는 게임 RTL과 이를 감싸는 wrapper(*RTL을 감싸서 Pico와 주고받는 8비트 데이터를 RTL의 입출력 신호로 변환해주는 FPGA 쪽 인터페이스 모듈)가 올라간다. wrapper는 Pico에서 8비트 버스로 받은 바이트를 btn_fire, btn_right, btn_left, rst_n으로 나누어 RTL에 넣고, RTL의 출력인 clk_o, lcd_data, v_sync를 한 바이트로 묶어 돌려준다. Q.14는 출력이 3비트로 줄어, 한 번에 주고받는 출력 바이트도 Q.13의 2개에서 1개로 줄었다.

Makefile은 이 세 부분을 단계별로 빌드한다. Step 1(make build_mi)은 arduino-cli로 Pico 펌웨어를 컴파일하고, Step 2(make build_trans)는 Quartus의 quartus_sh로 tcl 스크립트를 실행하여 FPGA 설정 파일(.sof)을 만들며, Step 3(make build)는 clang++로 PC 쪽 테스트벤치를 컴파일한다. Q.13과 Q.14 모두 세 단계의 빌드가 성공하였다. 펌웨어는 약 72KB로 Pico 플래시의 3%를 사용하였다. FPGA 빌드는 각 프로젝트의 최종 RTL(Q.13은 4_Ok, Q.14는 4_v_3)로 실행하였고 약 85초가 걸렸으며, quartus_cpf로 .sof 파일을 FPGA에 올리는 .rbf 파일로 변환하는 단계까지 확인하였다. 보드에 올리는 make config_trans는 openFPGALoader를 사용하는데, 아직 설치되어 있지 않아 보드를 받은 뒤 확인할 예정이다.

표 2. FPGA 자원 사용량 (Cyclone IV E EP4CE6E22C8, wrapper 포함)

항목	Q.13 (4_Ok)	Q.14 (4_v_3)
논리 소자 (LE) (FPGA내의 작은 회로 블록)	903 / 6,272 (14%)	683 / 6,272 (11%)
레지스터 (값을 저장하는 칸. 플립플롭)	180	122
메모리 비트, 곱셈기 (FPGA에 따로 들어 있는 전용블록)	0, 0	0, 0
최대 동작 주파수 (DUT 클럭) (회로가 오류없이 돌 수 있는 가장 빠른 클럭)	70.45 MHz	102.28 MHz

*Flip-Flop: 1비트 값을 저장하는 가장 작은 메모리 소자. 클럭이 뛰는 순간에만 입력값을 받아 저장하고, 다음 클럭까지 값을 유지함. (여러 개의 플립플롭을 묶은 것을 register라고 함) – 게임에서는 매 클럭마다 조합회로(AND, OR 등)가 값을 계

산하고, 플립플롭이 그 값을 저장하는 일을 반복하며 진행됨.

Q.14는 Q.13보다 논리 소자가 24%, 레지스터가 58개 적었고, 최대 동작 주파수는 70.45MHz에서 102.28MHz로 높아졌다. 출력 레지스터(lcd_x, lcd_y, lcd_write_en)와 별도의 스캔 카운터, 점수 기능이 빠진 결과이다. 두 프로젝트 모두 FPGA 용량의 15% 이하를 사용하였고, 메모리 블록과 곱셈기 블록은 쓰이지 않았다. 스프라이트와 글꼴 데이터가 메모리가 아닌 논리 회로로 합성되었기 때문이다.

[5] ETRI050 작업 공간과 단계별 도구

ETRI050 디렉토리는 검증된 RTL을 ETRI 0.5μm 표준 셀로 합성하고 칩 레이아웃(GDS)을 만드는 작업 공간이다. Makefile의 각 목표는 qflow를 거쳐 표 3의 도구를 호출한다. 이번에는 [2]의 면적 검증을 위해 합성 단계까지 실행하였고, 배치 이후의 단계는 4주차 수업에서 진행한다.

표 3. ETRI050 Makefile의 목표별 도구

make 목표	도구	하는 일
synthesize	yosys	RTL을 ETRI 표준 셀의 연결(넷리스트)로 변환 (합성)
place	graywolf	셀을 칩 위에 배치
sta	OpenSTA	신호가 클럭 주기 안에 도착하는지 계산 (타이밍 분석)
route	qrouter	셀 사이를 금속 배선으로 연결
migrate	magic	배선 결과를 레이아웃 형식(.mag)으로 변환
lvs	netgen	레이아웃과 합성 회로의 연결이 같은지 대조
chip_top	magic, klayout	28핀 패드 프레임과 코어를 합쳐 GDS 생성
ETRI050/simulation	iverilog, SystemC	합성된 게이트 넷리스트로 다시 시뮬레이션

세 디렉토리의 Makefile을 비교하면, 같은 RTL 파일이 단계마다 다른 도구를 거쳐 서로 다른 결과물이 된다(표 4). 테스트벤치는 simulation과 emulation이 같은 파일을 공유하며, 합성 결과의 단위가 다르므로(yosys는 표준 셀 면적, Quartus는 논리 소자 수) [2]와 [4]의 수치는 각각의 기준으로 해석해야 한다.

표 4. 디렉토리별 Makefile의 주요 명령과 도구

디렉토리	주요 명령	도구	RTL이 변환되는 형태	테스트벤치
simulation	make lint, build, run, wave	Verilator, g++(SystemC), GTKWave	C++ 모델 (PC에서 계산)	SystemC 테스트벤치
emulation	make build_mi, build_trans, gen_rbf, build	arduino-cli, Quartus (quartus_sh, quartus_cpf), clang++	FPGA 회로 (.sof, .rbf)	simulation과 같은 파일 (EMULATED_CO_SIM)
ETRI050	make synthesize ~ lvs, size	qflow (yosys, graywolf, OpenSTA, qrouter, magic, netgen), klayout	표준 셀 넷리스트 → 레이아웃 (GDS)	iverilog + SystemC (게이트 수준)